
Algorithm 11: Bounded Backward Half-Search

Input:

- V . Set of nodes
- $E_{s,in}(v)$. Set of scheduled INBOUND edges to v : $(w, v, \text{dep}, \text{arr})$
- $E_{t,in}(v)$. Set of INBOUND transfer edges to v : (w, v, Δ)
- $\text{country}(v)$. Country associated with node v
- $R_{\text{back}}(v, b)$. Precomputed reverse reachable country sets
- $MTT(c)$. Precomputed minimum transit times
- $\text{bucket}(t)$. Time bucket mapping function
- MaxDuration . Maximum allowed elapsed time for the half-search (e.g., 840 mins)
- MinCountries . Minimum unique countries required to save the path (e.g., 6)
- GlobalMinTime . Minimum realistic time to visit a country (e.g., 25 mins)

Output:

- Labels_B . Map of nodes to a set of Pareto-optimal backward labels

```
1: Initialize  $\text{Labels}_B(v) \leftarrow \emptyset$  for all  $v \in V$ 
2: Initialize priority queue  $Q \leftarrow \emptyset$  // ordered by decreasing score
3: // Initialize labels at arrivals (End of the journey)
4: for all  $v \in V$  do
5:   for all scheduled edges  $(w, v, \text{dep}, \text{arr}) \in E_{s,in}(v)$  do
6:     // We initialize the state sitting at the arrival node 'v', having just finished our journey
7:      $L \leftarrow (\text{node} = v, t = \text{arr}, C = \{\text{country}(v)\}, \text{succ} = \text{null}, \text{end\_time} = \text{arr})$ 
8:      $\text{Labels}_B(v) \leftarrow \text{Labels}_B(v) \cup \{L\}$ 
9:     // Score favors paths with MORE countries and LATER departure times (less elapsed time)
10:     $\text{compute\_score}(L) \leftarrow (|L.C| \times 10000) + L.t$ 
11:     $Q \leftarrow Q \cup \{L\}$ 
12: // Main loop
13: while  $Q \neq \emptyset$  do
14:   extract  $L = (v, t, C, \text{succ}, \text{end\_time})$  with maximum score from  $Q$ 
15:    $\text{elapsed} \leftarrow \text{end\_time} - t$ 
16:    $\text{remaining\_half\_time} \leftarrow \text{MaxDuration} - \text{elapsed}$ 
17:   // — 1. GLOBAL HARD FLOOR PRUNING (Backward Variant) —
18:    $\text{needed} \leftarrow \text{MinCountries} - |C|$ 
19:   if  $\text{needed} > 0 \wedge \text{remaining\_half\_time} < (\text{needed} \times \text{GlobalMinTime})$  then
20:     continue
21:   // — 2. TIME-BUDGETED UB PRUNING (Backward Variant) —
22:   if  $\text{needed} > 0$  then
23:      $b \leftarrow \text{bucket}(t)$ 
24:      $U \leftarrow R_{\text{back}}(v, b) \setminus C$ 
25:     Sort  $U$  as a sequence  $(c_1, c_2, \dots, c_k)$  such that  $MTT(c_1) \leq MTT(c_2) \leq \dots \leq MTT(c_k)$ 
26:      $\text{time\_spent} \leftarrow 0$ 
27:      $\text{UB} \leftarrow 0$ 
28:     for  $i = 1$  to  $|U|$  do
29:        $\text{time\_spent} \leftarrow \text{time\_spent} + MTT(c_i)$ 
30:       if  $\text{time\_spent} \leq \text{remaining\_half\_time}$  then
31:          $\text{UB} \leftarrow \text{UB} + 1$ 
32:       else
```

```

33:   |   |   |   |   | break
34:   |   |   |   |   | if  $|C| + \text{UB} < \text{MinCountries}$  then
35:   |   |   |   |   |   | continue
36: // — 3. EXTEND BACKWARD VIA SCHEDULED EDGES —
37: // We look for trains that ARRIVED at our current node BEFORE our current time 't'
38: for all  $(w, v, \text{dep}, \text{arr}) \in E_{s, \text{in}}(v)$  do
39:   | if  $\text{arr} \leq t$  then
40:     | new_elapsed  $\leftarrow \text{end\_time} - \text{dep}$ 
41:     | if new_elapsed  $\leq \text{MaxDuration}$  then
42:       |  $C' \leftarrow C \cup \{\text{country}(w)\}$ 
43:       | // The new state is sitting at node 'w' at time 'dep'
44:       |  $L' \leftarrow (w, \text{dep}, C', \text{succ} = L, \text{end\_time} = \text{end\_time})$ 
45:
46:       | // STRICT BACKWARD PARETO DOMINANCE: t must be  $\geq$  to be better
47:       | if  $\nexists L'' \in \text{Labels}_B(w)$  such that
48:         |  $(L''.t \geq \text{dep} \wedge L''.C \supseteq C' \wedge (L''.t > \text{dep} \vee L''.C \neq C'))$  then
49:           | remove all  $L'' \in \text{Labels}_B(w)$  such that
50:             |  $(\text{dep} \geq L''.t \wedge C' \supseteq L''.C \wedge (\text{dep} > L''.t \vee C' \neq L''.C))$ 
51:             |  $\text{Labels}_B(w) \leftarrow \text{Labels}_B(w) \cup \{L'\}$ 
52:             | compute score( $L'$ )
53:             |  $Q \leftarrow Q \cup \{L'\}$ 
54:
55: // — 4. EXTEND BACKWARD VIA TRANSFERS —
56: for all  $(w, v, \Delta) \in E_{t, \text{in}}(v)$  do
57:   |  $t_{\text{new}} \leftarrow t - \Delta$  // Subtract walking time to find when we needed to start walking
58:   | new_elapsed  $\leftarrow \text{end\_time} - t_{\text{new}}$ 
59:   | if new_elapsed  $\leq \text{MaxDuration}$  then
60:     |  $L' \leftarrow (w, t_{\text{new}}, C, \text{succ} = L, \text{end\_time} = \text{end\_time})$ 
61:
62:     | // Strict Backward Pareto Dominance
63:     | if  $\nexists L'' \in \text{Labels}_B(w)$  such that
64:       |  $(L''.t \geq t_{\text{new}} \wedge L''.C \supseteq C \wedge (L''.t > t_{\text{new}} \vee L''.C \neq C))$  then
65:         | remove all  $L'' \in \text{Labels}_B(w)$  such that
66:           |  $(t_{\text{new}} \geq L''.t \wedge C \supseteq L''.C \wedge (t_{\text{new}} > L''.t \vee C \neq L''.C))$ 
67:           |  $\text{Labels}_B(w) \leftarrow \text{Labels}_B(w) \cup \{L'\}$ 
68:           | compute score( $L'$ )
69:           |  $Q \leftarrow Q \cup \{L'\}$ 
70:
71: // — 5. CLEANUP —
72: for all  $v \in V$  do
73:   | for all  $L \in \text{Labels}_B(v)$  do
74:     | if  $|L.C| < \text{MinCountries}$  then
75:       | remove  $L$  from  $\text{Labels}_B(v)$ 
76: return  $\text{Labels}_B$ 

```